

Cloud Computing

Final Project

Docker and Kubernetes

Project Designer:

Mr. Hosseini

Course Instructor:

Dr. Javadi

Final Deadline:

January 31, 2024 – 23:59

Introduction

The goal of this exercise is to work with Docker and Kubernetes in a proper and structured way. Therefore, we aim to implement a relatively simple project using Docker in **minikube** and then deploy it on **Kubernetes**.

To complete this exercise, you must have Docker installed on your system. You can use the following links:

- <https://docs.docker.com/get-docker/>
 - <https://minikube.sigs.k8s.io/docs/start/>
-

Step 1

In this step, we will develop an **HTTP system monitoring** project. The task of this system is to **check the health status of web servers**.

Each user can register the desired website address in this system, so that it is checked periodically over specified time intervals.

Your server must support the following three endpoints:

- **POST:** `/api/server`
- **GET:** `/api/server?id=`

- **GET: /api/server/all**

For simplicity, **no authentication** is required. A user can add a web server by sending a **POST** request to the first endpoint. Your server should store the submitted address in a **database** and return an **id** to the user as the identifier.

The server should then, at intervals defined in the config (e.g., every 60 minutes), fetch the list of registered web servers and check their health via an HTTP GET request.

If the web server responds with a **200 success code**, the **success** counter is incremented by one. Otherwise, **failure** is incremented and the **last_failure** timestamp is updated.

The format should be something like this:

id	address	success	failure	last_failure	created_at
1	216.239.38.120	20	2	1703852574	1703840374
2	aut.ac.ir	30	50	1702840374	1700840374

The other two endpoints are for viewing the stored results:

- By providing the **id**, a user can view the status of the selected server.
- Or, by calling the last endpoint, the status of **all registered web servers** can be retrieved.

Your responses should be in **readable format**, preferably in **JSON**.

Important Note:

The project must be **configurable** (similar to assignment 2). The following fields should be configured via a config file:

- The port number for the server
- Time interval for health checks (either as cron or minutes)
- Separate **read** and **write** addresses for the database server (both initially the same; useful for StatefulSet)
- Database username and password

You can implement the project in any programming language.

Step 2

After implementation, write a **Dockerfile** to containerize your project.

Then, build the Dockerfile and **push the image to DockerHub**.

Important Note:

You **must use multistage build**. In the **first stage**, only build your project and produce the executable file. Then, in the **second stage**, run this file in an **Alpine** container.

Then write a full **docker-compose file**, including volumes and networks, for automating the creation of resources and dependencies, and build and run the project with it.

Include the following in your report:

1. Build the image using your Dockerfile
 2. Push the image to DockerHub and show the result
 3. (Optional but recommended) Show screenshots if you tested your image locally
 4. Demonstrate proper execution of docker-compose and its stages
-

Step 3

Now, deploy your project to **minikube** using **Kubernetes manifest files**.

Start with a **Secret** to store the **DB username and password**. Since this is sensitive data, it must be stored securely.

Then write a **StatefulSet** to manage the database (you can choose the database type). Make sure it uses the password from the Secret. Define separate **services** for **read** and **write** operations. Correct use of these will be graded.

You must also define **reasonable resource usage (request and limit)** for the database.

Then, create a **ConfigMap** for reading port, monitoring interval, and DB addresses.

Write a **Deployment** to manage the pods. Set the number of replicas to **2**. Inject the previously created Secret and ConfigMap into the pods.

Also, define proper **resource requests and limits** for your application.

Lastly, define a **Service** to make the application accessible.

Use `kubectl apply` in order to create the resources in **minikube**.

Include the following in your report:

1. Output of `kubectl get` showing resource creation
 2. IP addresses of pods and how services route traffic to them
 3. How many pods were created for the database StatefulSet? Explain your reasoning
 4. How master and replicas are used
-

Step 4

Now prepare a **Helm chart** to simplify and formalize the deployment process.

You should create the proper structure and correctly fill in the `values.yaml` file.

Include the following in your report:

1. Install the project using `helm install`
 2. Show correct resource creation and then delete using `helm uninstall`
-

Bonus Tasks

These tasks were not fully covered in class. You must research them yourself (especially recommended for the first one).

1. Horizontal Pod Autoscaler (HPA) – (20%)

Deploy an **HPA** to automatically scale your app.

You must also deploy a **metrics server**.

In your report:

- List HPA parameters used for scaling
- Which parameter(s) did you use and why?

- Show HPA YAML or command used
 - Demonstrate that scaling happens under load
-

2. Probes: liveness, readiness, startup – (30%)

Define these probes correctly.

In your report:

- Explain how each probe works and its effect
 - Justify your design decisions
-

3. Prometheus Metrics – (30%)

Use **Prometheus** to define metrics like:

- Number of requests
- Number of failed requests
- Request latency
- Database error count
- DB response time

In your report:

- List metrics and their types
 - Provide one query for each collected metric
-

4. Grafana Dashboards – (20%)

Use **Grafana** to create dashboards to visualize the Prometheus metrics.

In your report:

- List the queries used in each dashboard
 - Include screenshots of all dashboards
-

Project Testing

Test your deployed project by **port forwarding the service** and include the results in your report.

Project Submission Guidelines

- The project will be evaluated via **Skype interview**, so avoid using code you can't explain.
 - You'll be asked about your Kubernetes files, their relationships, and making changes live via updated config.
 - Post your questions on the class website or Telegram group for a fast response.
-

What You Must Submit

A zip file named: `GID_FinalProject.zip` containing:

- All project files (in a folder)
- The project report (in a separate folder)